

Tres métodos de implementación

En los módulos 1 y 2, aprendiste a ejecutar agentes con **adk web**, que es la interfaz web visual para el desarrollo y las pruebas. Usaste comandos como el siguiente:

```
adk web
```

Este comando abre una IU basada en el navegador en la que puedes chatear con tu agente de forma interactiva.

Sin embargo, **adk web** no es la única forma de ejecutar agentes. En este módulo, descubrirás tres métodos adicionales:

- **adk run**: Para una ejecución basada en la terminal
- **adk api_server**: Para implementar como un servicio de API
- **Ejecución programática**: Para integrar agentes en aplicaciones de Python

Método 1: Ejecución en la terminal con **adk run**

Referencia: [Documentos del ADK: Ejecuta tu agente](#)

En qué consiste

El comando **adk run** te permite interactuar con tu agente directamente desde la terminal, sin abrir un navegador web.

Instrucciones de uso

Desde el directorio de tu agente:

```
# Navega al directorio de tu proyecto de agente
cd my_first_agent

# Ejecuta el agente
adk run
```

Comportamiento esperado:

- La terminal se vuelve interactiva.
- Escribe el mensaje y presiona la tecla **Intro** en el teclado.
- El agente responderá en la terminal.
- Escribe otro mensaje para continuar la conversación.
- Presiona **Ctrl + C** para salir.

Desde el directorio superior:

```
# Si estás en el directorio superior (p. ej., adk-workspace)
adk run my_first_agent
```

Interacción de ejemplo

```
$ adk run my_first_agent
```

Tú: ¿Cómo **resuelvo** $x + 5 = 10$?

Agente: ¡Excelente pregunta! Trabajemos juntos para resolver este problema. ¿Qué **crees** que deberíamos hacer para **despejar** la x ?

Tú: ¿Restar **5** de ambos lados de la ecuación?

Agente: ¡Exactamente! Ese es el método correcto. Cuando restamos **5** de ambos lados de la ecuación...

Cuándo deberías usar **adk run**

Es ideal para estos casos:

-  Pruebas rápidas durante el desarrollo
-  Flujos de trabajo de línea de comandos
-  Entornos de servidor sin GUI
-  Secuencias de comandos de pruebas automatizadas
-  Canalizaciones de CI/CD

Casos de uso inadecuados:

-  Presentar a las partes interesadas (en su lugar, usa **adk web**)
-  Depurar conversaciones complejas (usa **adk web** en su lugar)

Método 2: Servidor de API con **adk api_server**

Referencia: [Documentos de ADK - Configuración del agente: Ejecutar](#)

En qué consiste

El comando **adk api_server** ejecuta tu agente como un servicio de API de REST, lo que permite que otras aplicaciones envíen solicitudes a tu agente a través de HTTP.

Instrucciones de uso

Inicia el servidor de la API:

```
# Desde el directorio de tu agente
cd my_first_agent
adk api_server
```

O desde el directorio superior:

```
adk api_server my_first_agent
```

Resultado esperado:

```
INFO:      Started server process [12345]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Pruebas con cURL

Una vez que el servidor esté en ejecución, puedes enviar solicitudes con `curl`:

```
# En una ventana de terminal distinta
curl -X POST http://localhost:8000/your-endpoint \
  -H "Content-Type: application/json" \
  -d '{"message": "¿Cuánto es 2x + 5= 13?"}'
```

Nota: Para obtener información detallada sobre el uso de la API, consulta la [documentación de pruebas del ADK](#).

Cuándo usar `adk api_server`

Es ideal para estos casos:

-  Integrar agentes en aplicaciones web
-  Backends de apps para dispositivos móviles
-  Arquitecturas de microservicios
-  Pruebas de preproducción
-  Desarrollos de APIs locales antes de implementar en Cloud Run

Casos de uso inadecuados:

-  Desarrollo interactivo (usa `adk web` en su lugar)
-  Pruebas rápidas (usa `adk run` en su lugar)

Método 3: Ejecución programática con Python

Referencia: [Documentos del ADK - Instructivo del equipo de agentes](#)

En qué consiste

Puedes ejecutar agentes directamente en tu código de Python, lo que te da un control programático completo. Este método es útil en estos casos:

- Creación de aplicaciones personalizadas con agentes
- Notebooks de Jupyter/Google Colab
- Canalizaciones de procesamiento de datos
- Integraciones personalizadas

Ejemplo completo (listo para copiar y pegar)

Este ejemplo es independiente y está listo para ejecutarse en una secuencia de comandos de Python o en un notebook de Jupyter:

```
"""
Ejemplo completo: Ejecuta un agente del ADK de forma programática
Copia todo este bloque de código para ejecutarlo en una secuencia de comandos de
Python o en un notebook.
"""

# Paso 1: Instala el ADK (ejecuta esto en la terminal o celda del notebook)
# pip install google-adk

# Paso 2: Establece tu clave de API
# Opción A: Configurarla como variable de entorno antes de ejecutar
# export GOOGLE_API_KEY=your-api-key-here
# Opción B: Quitar el comentario y usar este código:
# import os
# os.environ['GOOGLE_API_KEY'] = 'your-api-key-here'
# os.environ['GOOGLE_GENAI_USE_VERTEXAI'] = 'FALSE'

# Paso 3: Importa las bibliotecas necesarias
import asyncio
from google.adk.agents.llm_agent import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Paso 4: Define tu agente
agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor',
    instruction="""Eres un tutor de matemáticas paciente.
    Guía a los estudiantes en la resolución de problemas paso a paso.
    No te limites a dar respuestas, ayúdalos a descubrir las soluciones."""
)

# Paso 5: Configura la sesión y el ejecutor
APP_NAME = "math_tutor_app"
USER_ID = "student_1"
SESSION_ID = "session_001"

session_service = InMemorySessionService()
runner = Runner(
    agent=agent,
    app_name=APP_NAME,
```

```

        session_service=session_service
    )

# Paso 6: Define la función asíncrona para ejecutar el agente
async def run_agent():
    # Crea una sesión
    session = await session_service.create_session(
        app_name=APP_NAME,
        user_id=USER_ID,
        session_id=SESSION_ID
    )
    print(f"Sesión creada: {SESSION_ID}\n")

    # Prepara el mensaje del usuario
    user_message = Content(
        role="user",
        parts=[Part(text="¿Cuánto es 2x + 5= 13?")]
    )

    # Ejecuta el agente y recopila la respuesta.
    print("Usuario: ¿Cuánto es 2x + 5= 13?\n")
    print("Agente: ", end="")

    async for event in runner.run_async(
        user_id=USER_ID,
        session_id=SESSION_ID,
        new_message=user_message
    ):
        # Imprime la respuesta final
        if event.is_final_response() and event.content and event.content.parts:
            print(event.content.parts[0].text)

# Paso 7: Ejecuta el agente
# Para Jupyter/Colab, usa el operador await directamente
# await run_agent()

# Para secuencias de comandos de Python, usa asyncio.run()
asyncio.run(run_agent())

```

Ejecuta este ejemplo

En Jupyter Notebook o Google Colab:

```

# Simplemente usa el operador await (el bucle de evento ya se está ejecutando)
await run_agent()

```

En una secuencia de comandos de Python:

```

# Usa asyncio.run() para iniciar el bucle de evento
asyncio.run(run_agent())

```

Resultado esperado:

Sesión creada: session_001

Usuario: ¿Cuánto es $2x + 5 = 13$?

Agente: ¡Excelente pregunta! Trabajemos juntos para resolver este problema.
Primero, ¿Qué crees
que hay que hacer para despejar la x ?

Cuándo usar una ejecución programática

Es ideal para estos casos:

-  Notebooks de Jupyter y Google Colab
-  Aplicaciones personalizadas de Python
-  Canalizaciones de procesamiento de datos
-  Investigación y experimentación
-  Control detallado de la ejecución

Casos de uso inadecuados:

-  Pruebas interactivas rápidas (usa `adk web` o `adk run`)
-  Entornos que no son de Python

Comparación: Cuándo usar cada método

Método	Casos de uso indicados	Persistencia de la sesión	Complejidad de configuración
<code>adk web</code>	Desarrollo, depuración, demostraciones	 Sí (en el navegador)	Baja
<code>adk run</code>	Pruebas rápidas, flujos de trabajo de CLI	 No	Muy baja
<code>adk api_server</code>	Integración de APIs, producción	Depende del cliente	Baja
Programático	Apps personalizadas, notebooks	 Sí (administrada)	Media

Resumen:

- Para el desarrollo, usa `adk web`.
- Para una prueba rápida, usa `adk run`.
- Para crear una API, usa `adk api_server`.
- Para una integración personalizada, usa la ejecución programática.

Conclusiones principales

Elige la herramienta adecuada:

- Usa `adk web` para el desarrollo visual y la depuración.
- Usa `adk run` para realizar pruebas rápidas en la línea de comandos.
- Usa `adk api_server` para implementar como una API.
- Usa la ejecución programática para aplicaciones personalizadas de Python.

Todos los métodos funcionan con el mismo agente:

- Tu archivo `agent.py` no cambia.
- Los diferentes métodos son solo formas distintas de interactuar.
- Elige uno en función de tus necesidades actuales de flujo de trabajo.